

Opérateurs OLAP

OLFA DRIDI



Opérateurs OLAP



Dans un cube



Sur plusieurs cubes



Extensions SQL



MDX

Plan

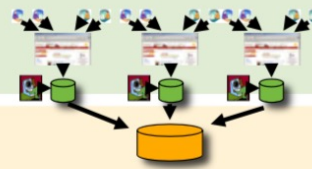
Rappels : Requêtes

3

BD opérationnelles

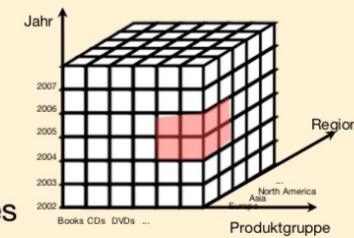
- Requêtes transactionnelles
- INSERT, UPDATE, DELETE, SELECT
- Requêtes portant sur quelques tuples
- Optimisation de requêtes “classique”

```
UPDATE Order
SET amount = amount + 1
WHERE OrderID = 1
AND BookID = 204
```



Entrepôt de données

- Requêtes analytiques
- *Bulk-Insert* et SELECT
- Requêtes typiques (*drill down*, *slice*, *dice*, ...)
- Techniques d'optimisation de requêtes spécialisées

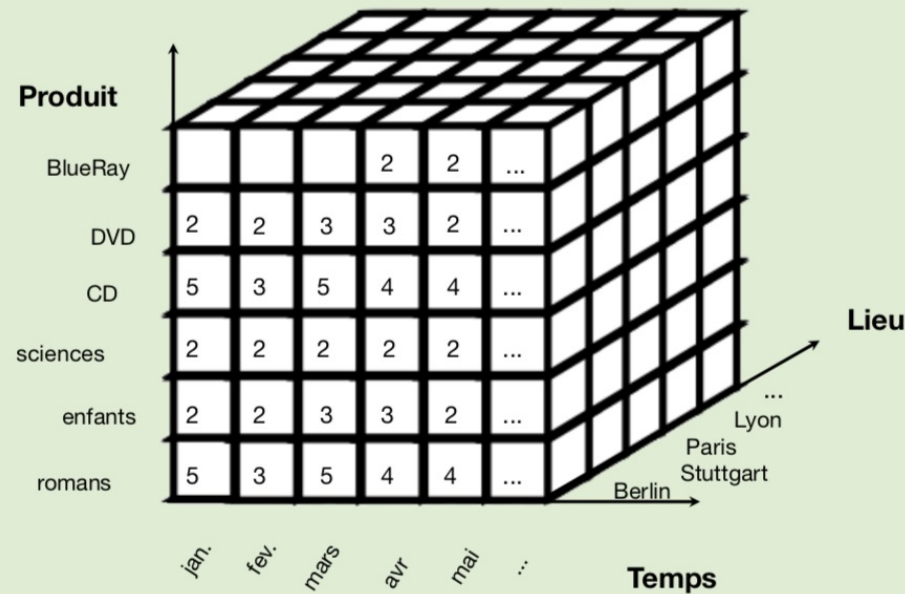


Un modèle multidimensionnelle

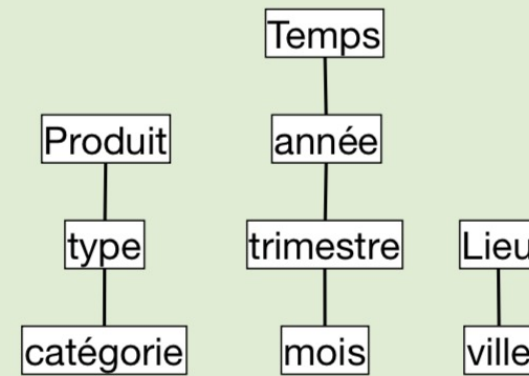
4

Exemple d'un modèle multidimensionnel (cube et hiérarchies dimensionnelles)

Cube (granularité la plus fine)



hiérarchies dimensionnelles



Objectif: Répondre à des requêtes analytiques

5

- ▶ **Rappel:** Le DW intègre des données pour obtenir une vue d'ensemble des processus d'entreprise faisant l'objet d'analyses.
- ▶ Contrairement aux requêtes sur des données relationnelles “classiques”, les requêtes analytiques suivent souvent un même schéma.
- ▶ On distingue ainsi entre des requêtes **OLTP (Online Transaction Processing)** posées à des systèmes opérationnels et des requêtes **OLAP (Online Analytical Processing)** posées à des entrepôts de données.

Exemples de requêtes OLAP

6

- ▶ **Navigation dans un cube**

“Nombre de livres d’enfants vendu en janvier, indépendamment du lieu”

- ▶ **Navigation à travers les niveaux de différente granularité**

“CD par type de produit (musique, film, ...) par trimestre et ville”

- ▶ **Navigation à travers plusieurs cubes corrélés**

“Ventes totales en magasin et en ligne”, partant de deux cubes (Produit/Temps/Lieu) et (Produit/Temps/Client).

- ▶ **Requêtes de *ranking***

“Les 10 livres les plus populaires en 2022 par pays”.

Dans ce chapitre, nous allons discuter les schémas typiques de requêtes OLAP et les langages développés afin de les spécifier.

Vue d'Ensemble des Requêtes OLAP

7

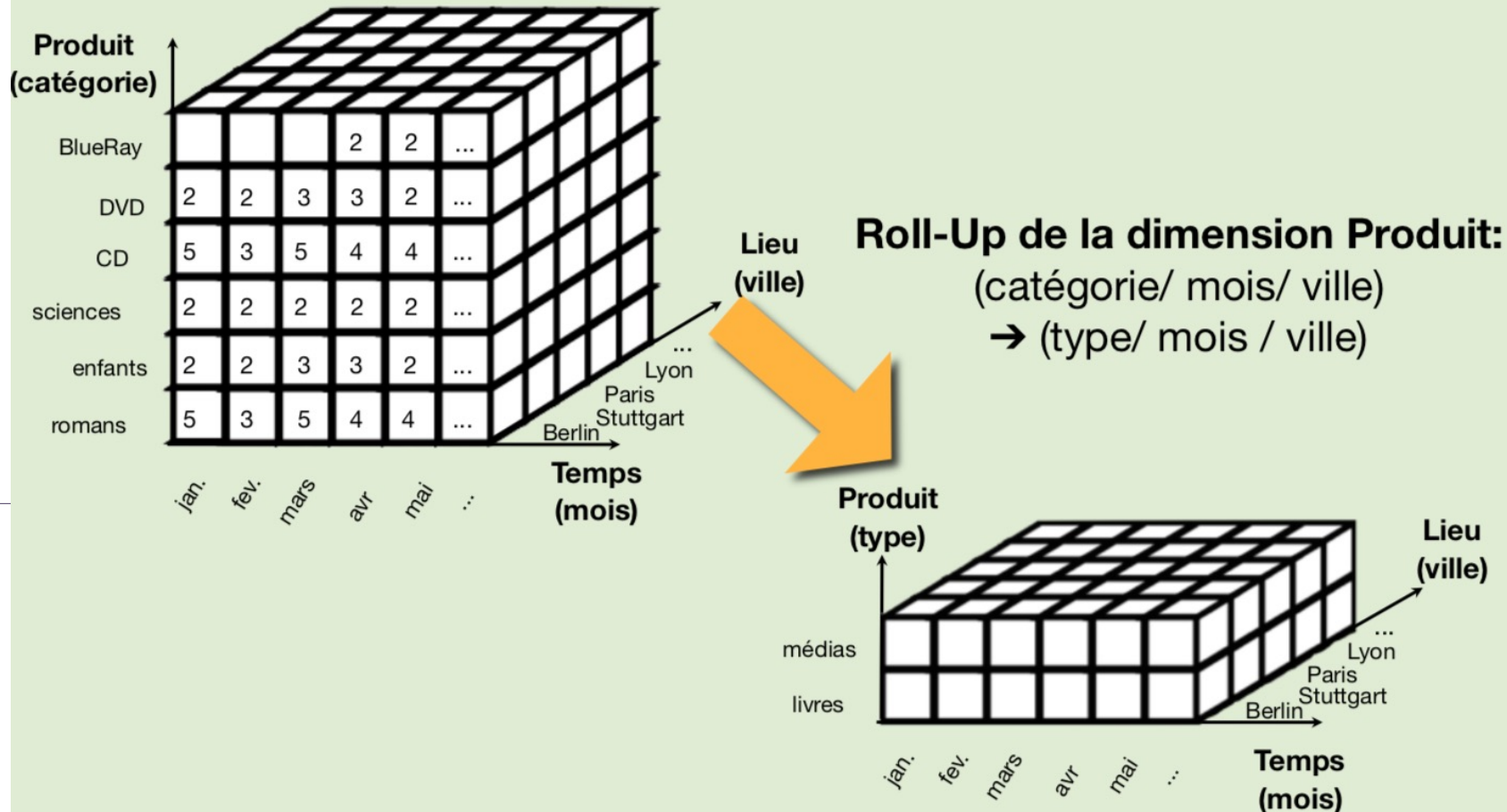
- ▶ Les requêtes abordées dans cette partie du chapitre permettent la **navigation** dans le modèle multidimensionnel (*MDDM – multi-dimensional data model*).
- ▶ **Roll-up / Consolidate**: naviguer vers une granularité plus grossière
- ▶ **Drill-down**: naviguer vers une granularité plus fine
- ▶ **Drill-out / Split**: ajouter une ou plusieurs dimension
- ▶ **Drill-in / Merge**: réduire le cube d'une ou de plusieurs dimensions
- ▶ **Slice**: sélectionner des données en appliquant un critère de sélection à une dimension.
- ▶ **Dice**: sélectionner des données en appliquant une sélection à plusieurs dimensions.

- ▶ Un roll-up correspond donc à un zoom-out du cube actuel, ce qui entraîne un résultat de granularité réduite.
Par exemple roll-up du cube (*catégorie*/ mois/ ville) vers(*type*/ mois/ ville).
- ▶ Cette opération peut porter sur une ou plusieurs dimensions à la fois
Par exemple roll-up du cube (*catégorie* / mois / ville) vers (*type*/ *trimestre*/ ville).
- ▶ Il est également possible de naviguer vers la racine TopD d'une ou de plusieurs dimensions. Appliquée à toutes les dimensions, cette opération résulte en un cube n'ayant plus qu'un seul fait.

Exemple Roll-Up d'une seule dimension

9

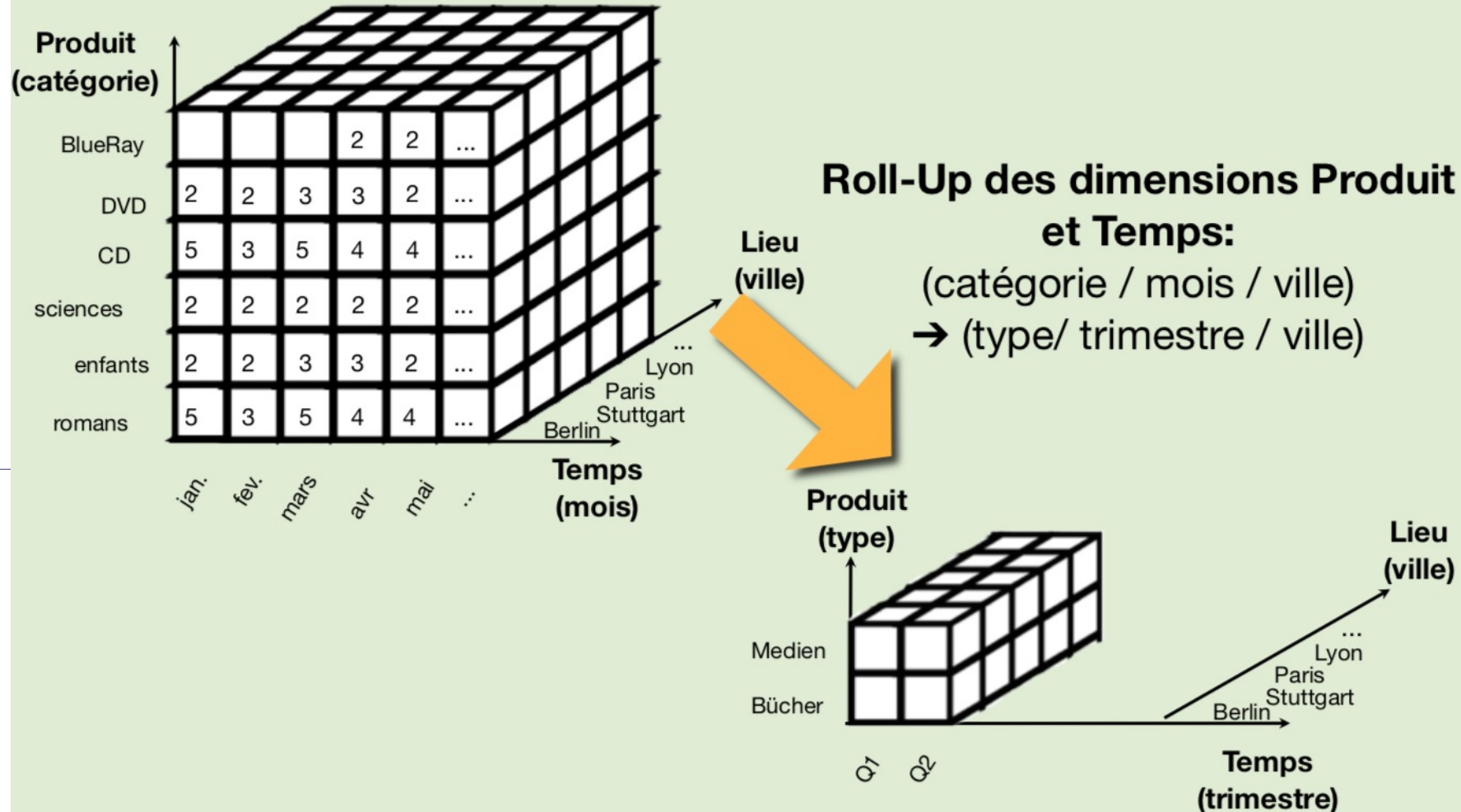
Roll-up d'une seule dimension



Exemple Roll-Up de plusieurs dimensions

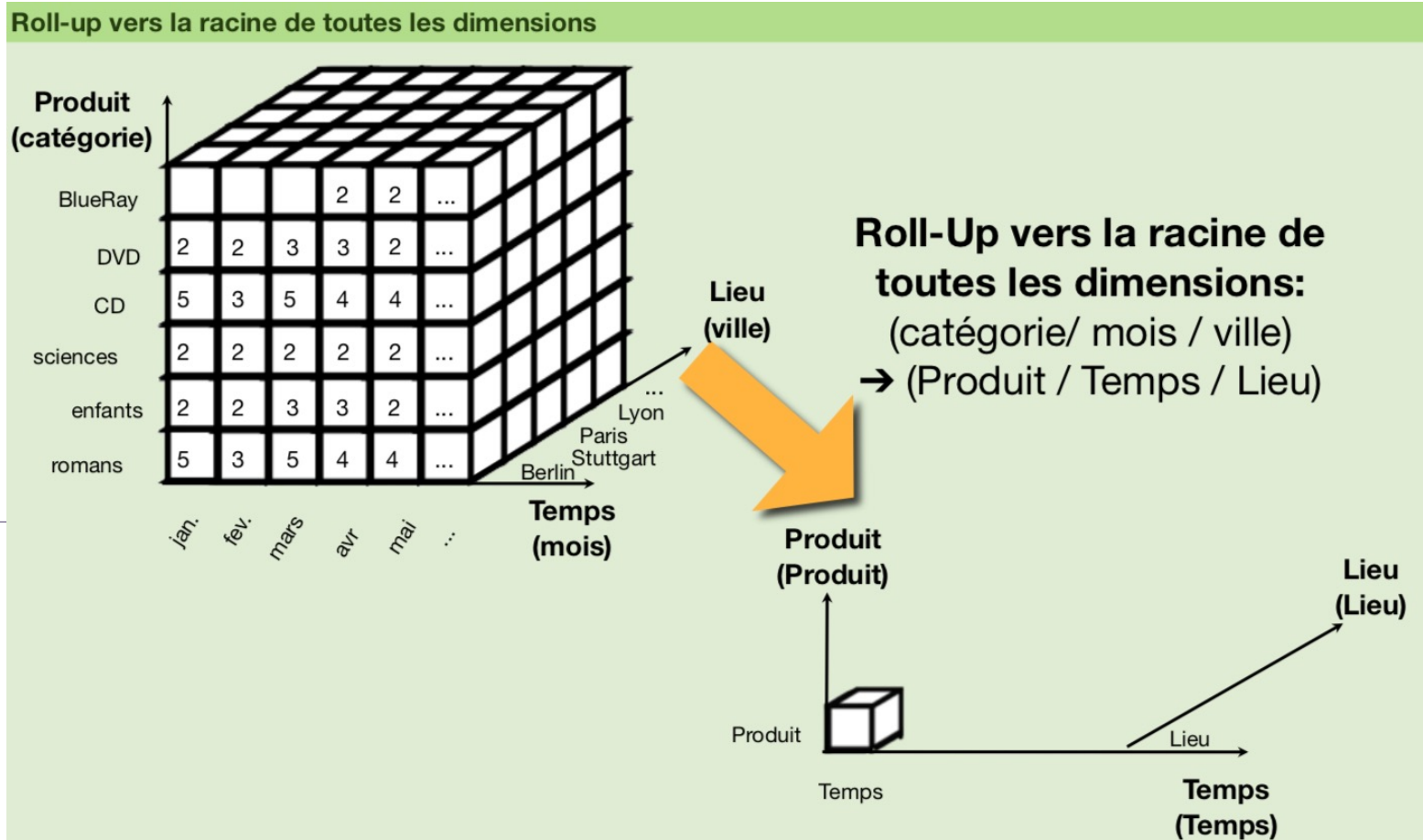
10

Roll-up de plusieurs dimensions



Exemple Roll-Up vers la racine de toutes les dimensions

11



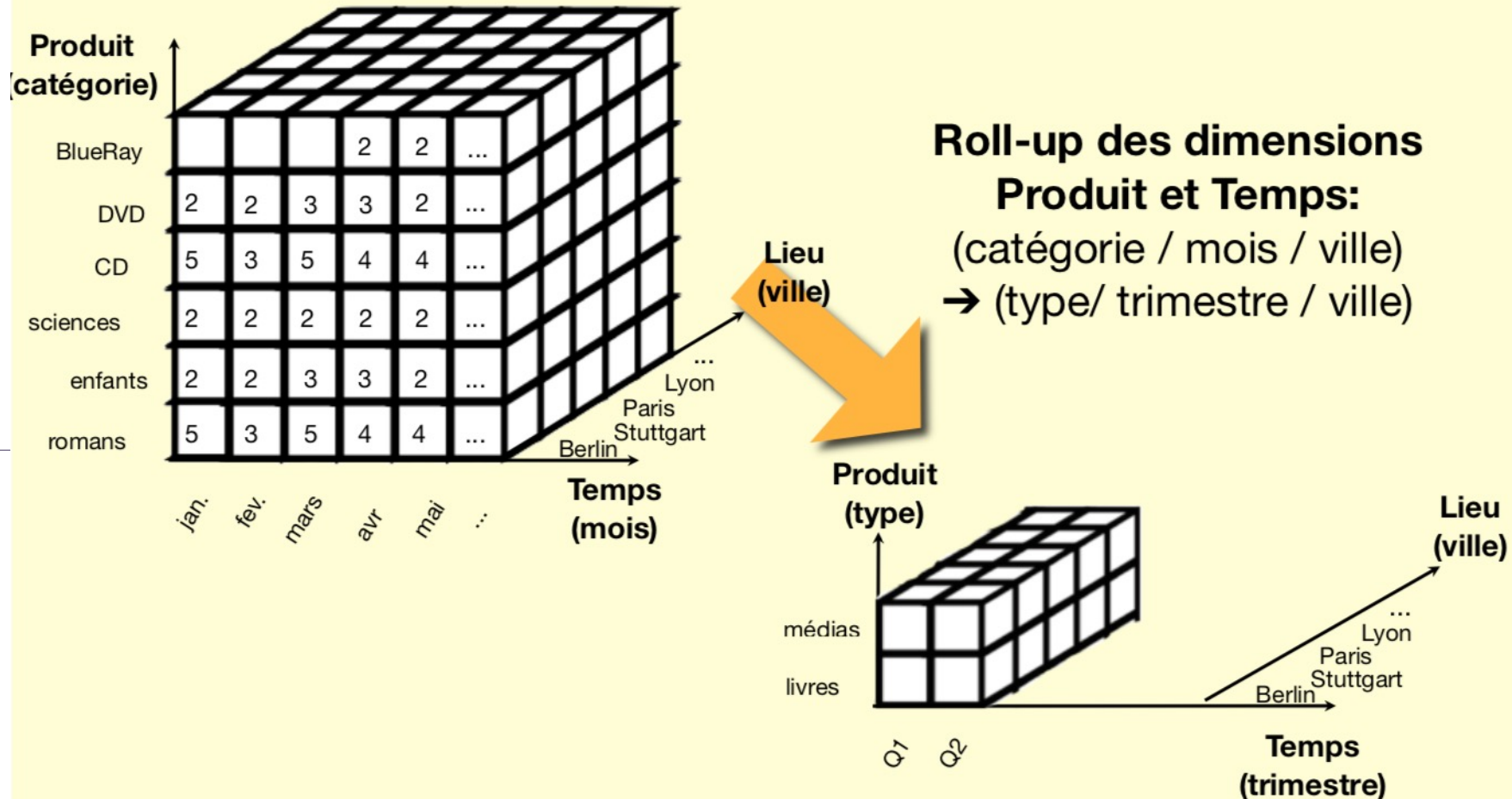
Rappel: Une mesure appartient à un fait, et ce fait possède une certaine granularité.

- ▶ Deux propriétés d'une mesure:
 - ▶ Valeur numérique
 - ▶ Fonction pouvant dériver la mesure à partir d'autres mesures.
- ▶ Lors d'une opération roll-up, nous passons d'une granularité $G1$ des faits vers une granularité plus grossière des faits, dénotée $G2$.
- ▶ La **fonction** définie pour les faits de granularité $G2$ (si celle-ci est définie) est utilisée pour calculer la valeur de la mesure de granularité $G2$.
- ▶ Cependant, il faut prendre en compte la propriété de **l'additivité** de la mesure (additive, semi-additive ou non-additive).
- ▶ Dans le cas d'une mesure semi-additive ou non-additive, il se peut que la valeur de la mesure ne soit calculable qu'à partir des mesures des données à plus fine granularité (de toutes les dimensions).

Roll-Up: Valeurs de mesures

13

Dérivez les valeurs des mesures du résultat de l'opération roll-up représentée ci-dessous (en utilisant les valeurs montrées uniquement)



Drill-down

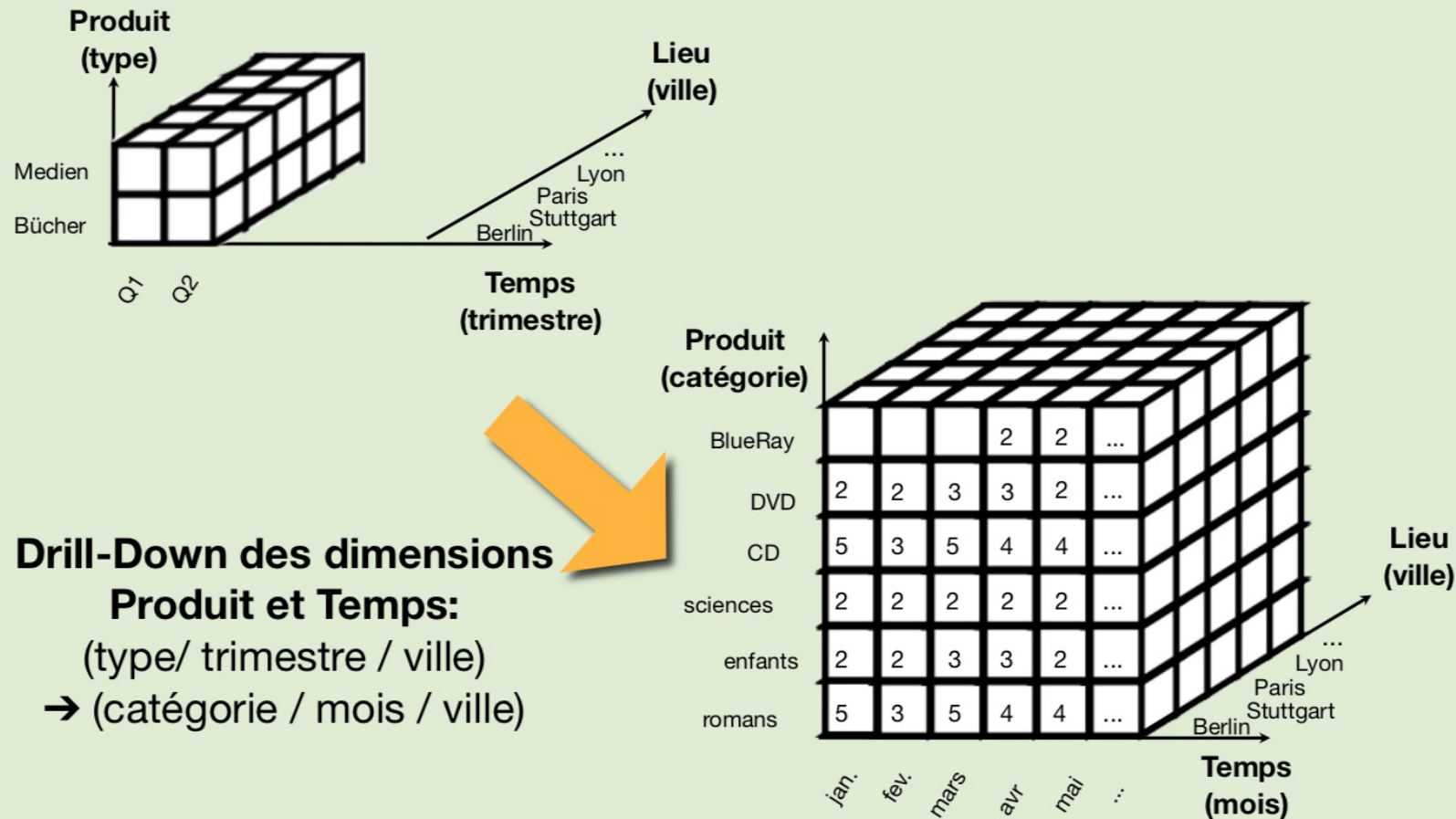
14

- ▶ Drill-down est l'opération **inverse de Roll-Up**.
- ▶ Rend possible la navigation d'une granularité donnée vers une granularité plus fine pour une ou plusieurs dimensions (zoom-in).
- ▶ Par exemple de (*type / trimestre / ville*) vers (*catégorie / mois / ville*).

Exemple: Drill-down de plusieurs dimensions

15

Drill-down de plusieurs dimensions



Récapitulatif Roll-Up / Drill-Down

16

- ▶ Opérateurs servant à **naviguer entre différentes granularités** d'une ou de plusieurs dimensions.
 - ▶ Roll-up: moins de détails
 - ▶ Drill-down: plus de détails
- ▶ Détermination des mesures (lorsque le cube résultat n'est pas matérialisé):
 - ▶ Roll-Up: possibilité de calcul à partir du cube d'entrée.
 - ▶ Drill-down: utilisation des mesures des faits de plus fine granularité (et qui sont matérialisés).
- ▶ Dans le deux cas, **le nombre de dimensions du cube reste inchangée**, c.a.d. l'application d'un roll-up / drill-down sur un cube n -dimensionnel résulte en un cube n -dimensionnel.
- ▶ Exception: navigation vers / depuis la racine d'au moins une dimension.

- ▶ Lors d'un **drill-out (aussi nommé *split*)** un cube à n dimensions est **complété par k dimensions** de granularité plus fine que TopD, où $k \geq 1$.
 - Le résultat est un cube à $(n+k)$ dimensions
- ▶ Comme drill-down, drill-out navigue vers des données **plus détaillées**.
- ▶ Contrairement à drill-down, le détail n'est pas augmenté par l'augmentation de la granularité d'une dimension existante, mais par l'ajout de nouvelles informations **d'une nouvelle dimension**.

Drill-Out / Split

18

Drill-out d'un cube à une dimension (catégorie) vers un cube 2D (catégorie / trimestre)

Produit (catégorie)	
BlueRay	30
DVD	40
CD	20
sciences	15
enfants	20
romans	50

Drill-out de la dimension temps (trimestre)



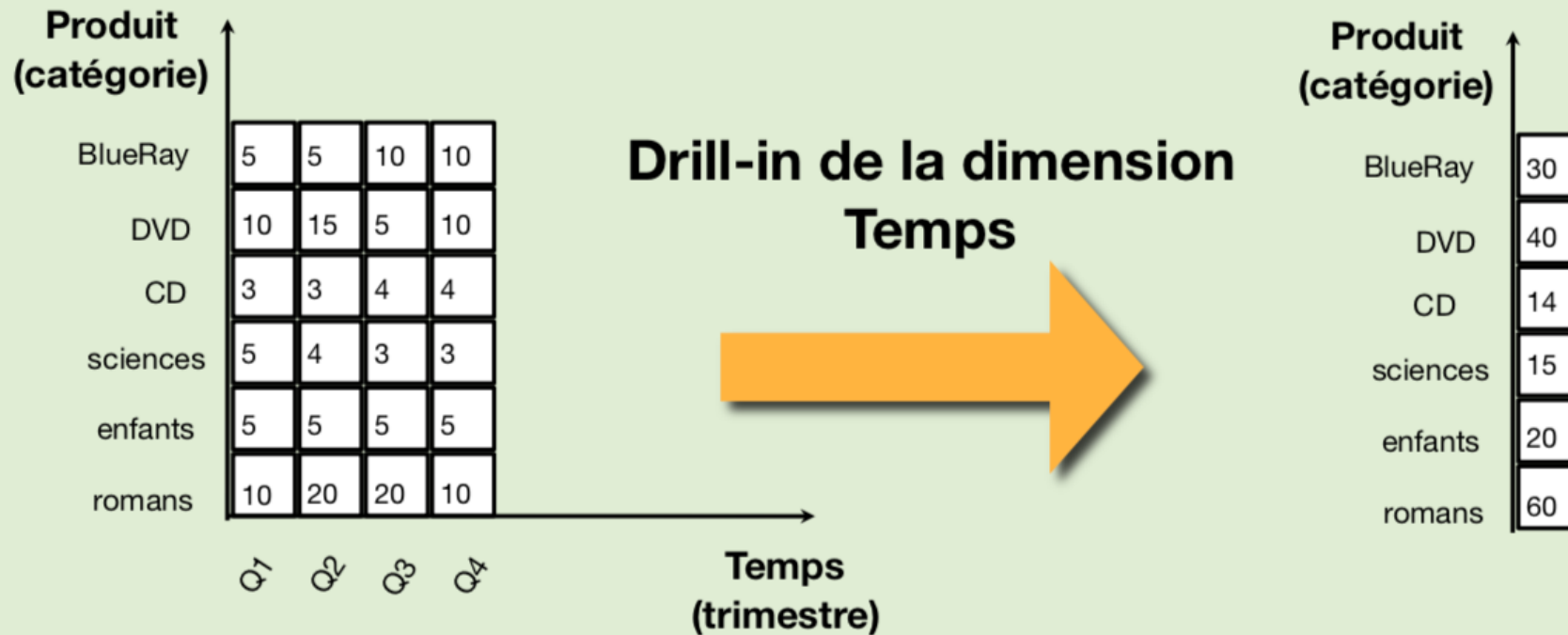
Produit (catégorie)				
BlueRay	5	5	10	10
DVD	10	15	5	10
CD	3	3	10	4
sciences	5	4	3	3
enfants	5	5	5	5
romans	10	20	10	10
	Q ₁	Q ₂	Q ₃	Q ₄
	Temps (trimestre)			

- ▶ Opération inverse de drill-out.
- ▶ Un **drill-in (nommé également merge)** efface **k dimensions** d'un cube n-dimensionnel, où $k < n$.
 - Le résultat est un cube à (n-k) dimensions.
- ▶ Comme pour roll-up, les données résultant d'un drill-in sont **moins détaillées** que les données d'entrée de l'opération.
- ▶ Contrairement au roll-up, le degré de détail n'est pas réduit par réduction de la granularité, mais par l'élimination de dimensions.

Drill-In / Merge

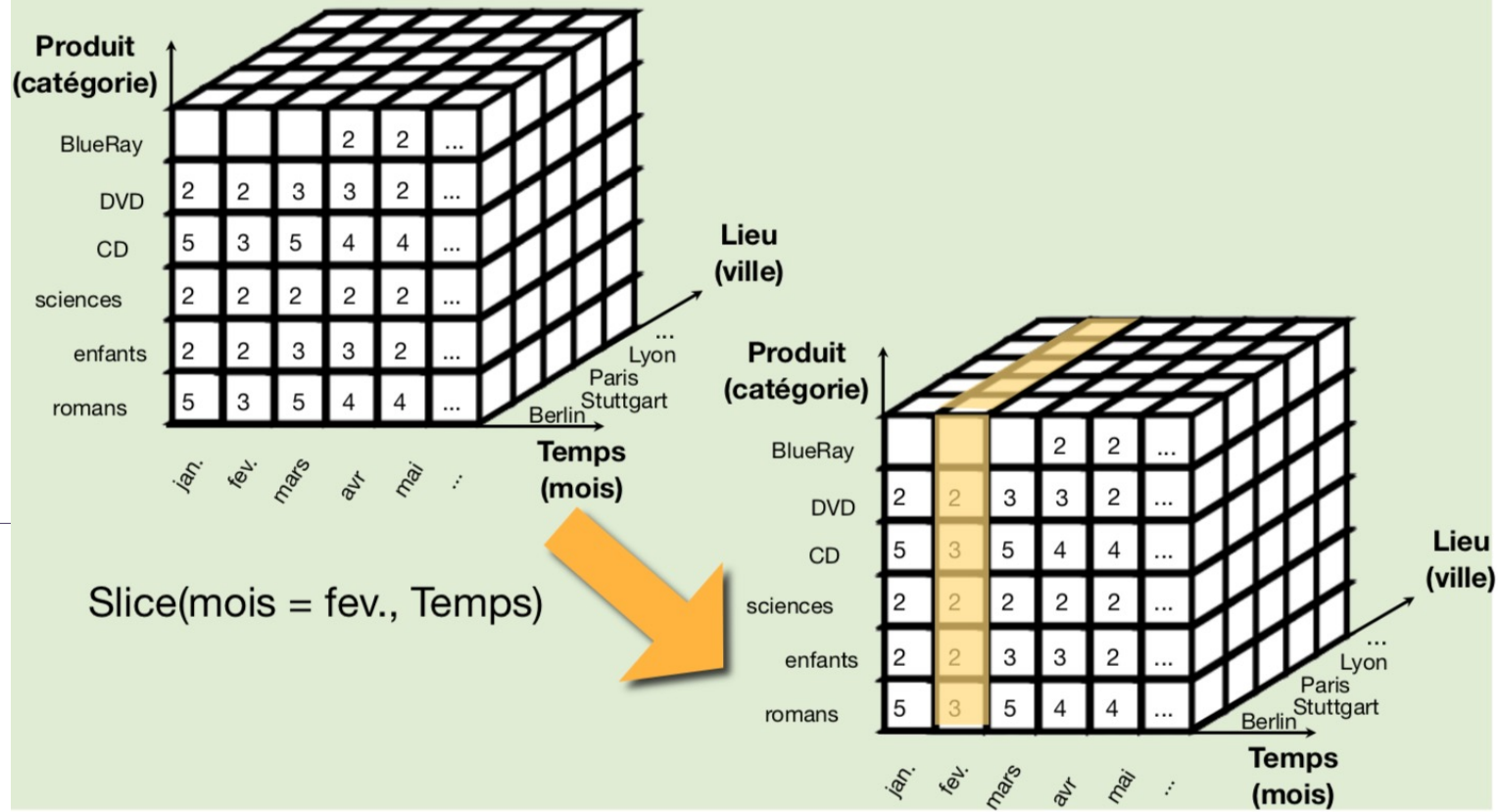
20

Drill-in du cube 2D (catégorie / trimestre) vers le cube à une dimension (catégorie)



- ▶ Permet l'obtention d'un **extrait** des informations représentées par un cube en fonction des termes définis pour **une dimension**.
- ▶ Le **paramètre** de l'opérateur *slice* est un **critère de sélection** (*range query, point query*) portant sur une dimension.
- ▶ Critère de sélection avec AND, OR et NOT.
Par exemple (mois > fev.) AND (mois <= nov.) AND (mois NOT avr)
- ▶ Critère de sélection sur un ou plusieurs niveaux d'une même dimension
Par exemple pays = DE AND code_postal= 10179
- ▶ Notation ***Slice(p, D)***: sélectionne les “tranches” le long de la dimension D, qui satisfont le critère de sélection p.

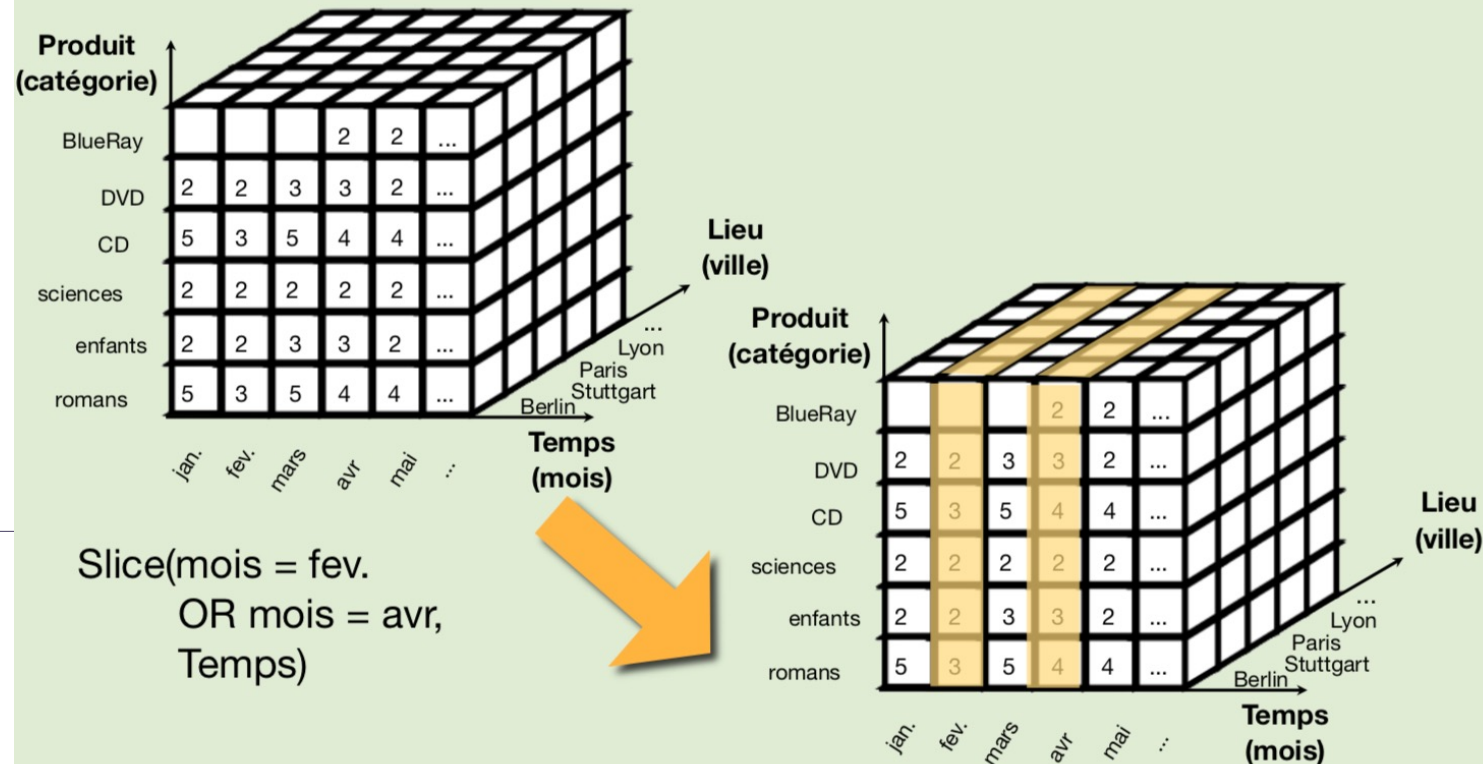
Slice avec critère de sélection simple (mois = fev.)



Slice

23

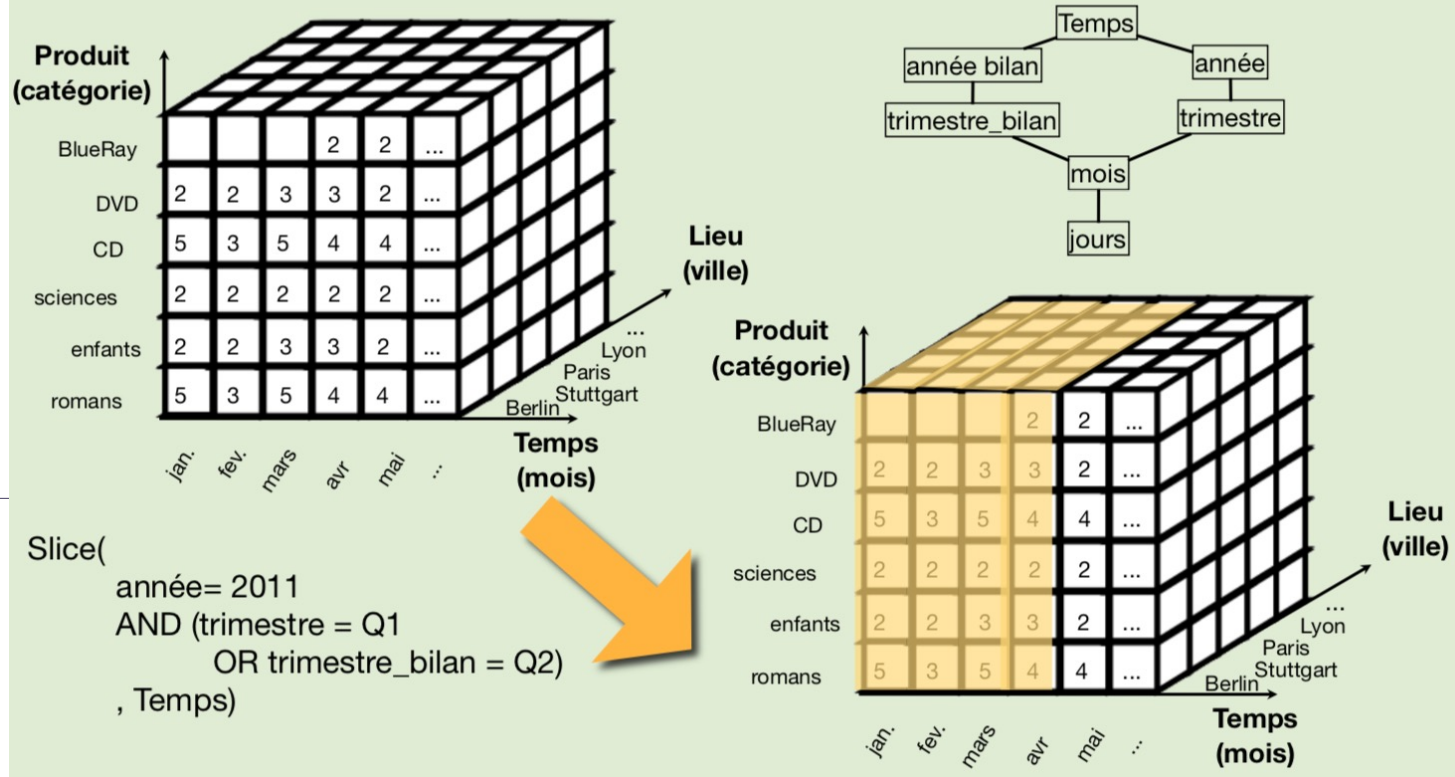
Slice avec critère de sélection contenant une disjonction (mois = fev. OR mois = avr)



Slice

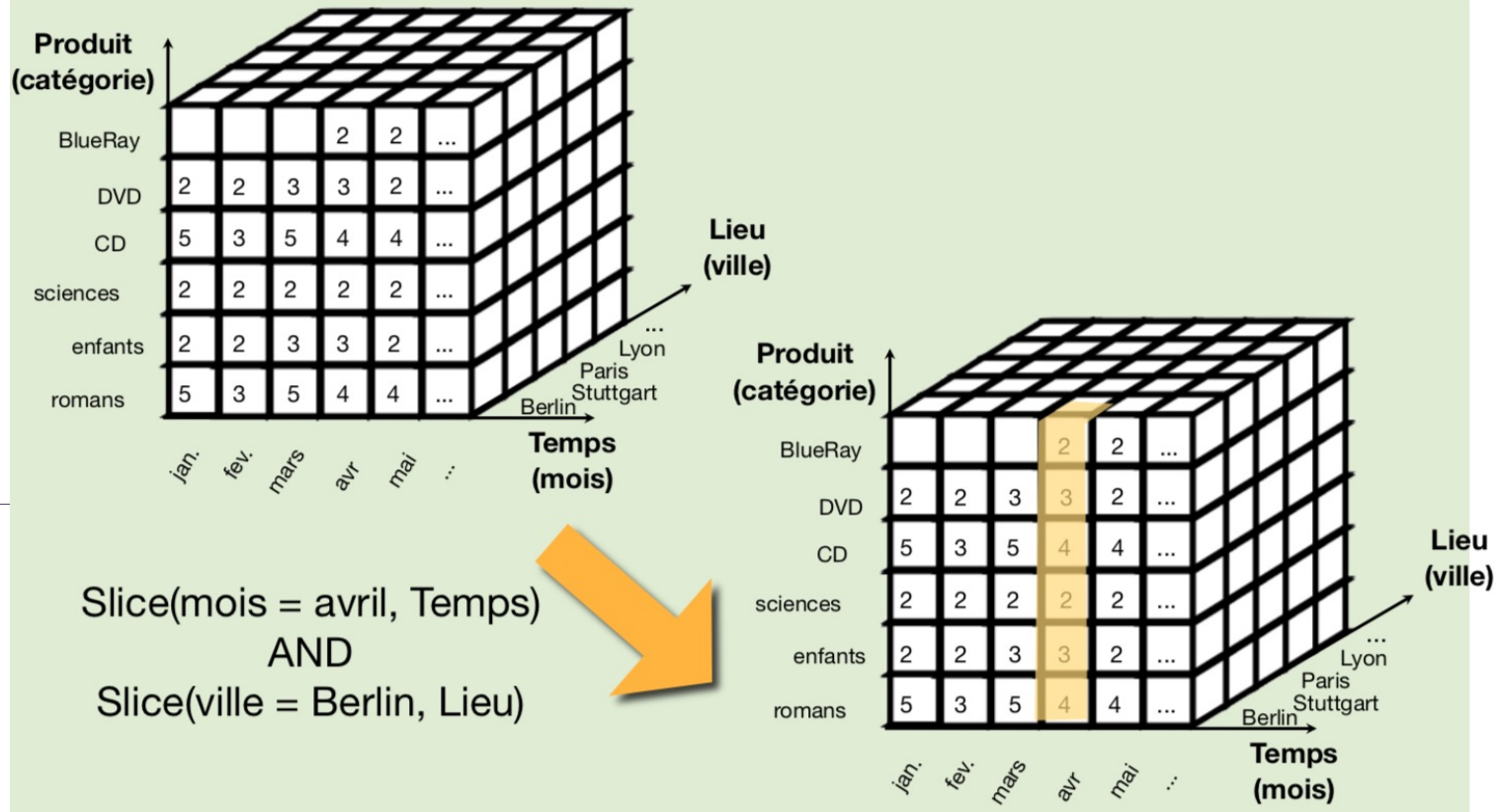
24

Slice portant sur la dimension Temps, qui se compose de deux hiérarchies

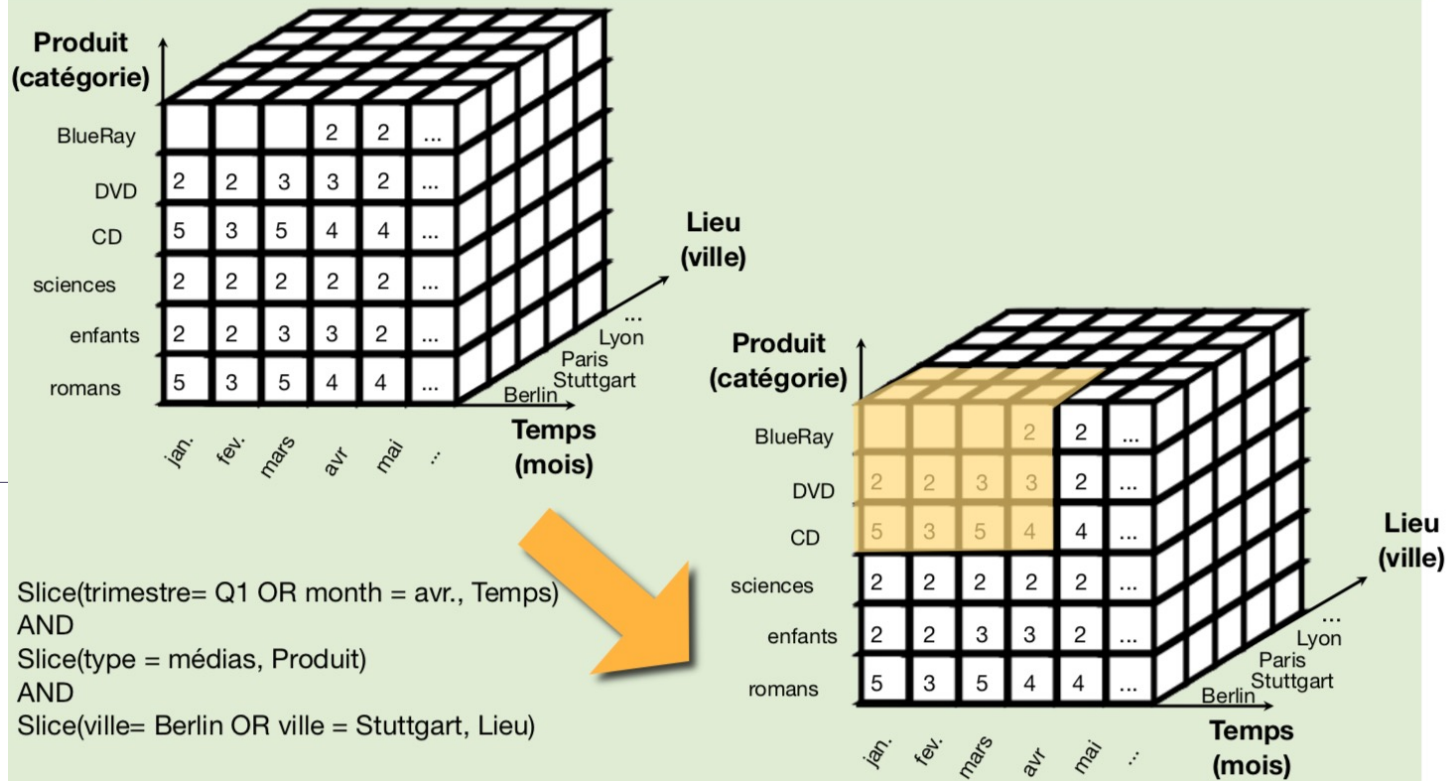


- ▶ Application de l'opérateur slice à plusieurs dimensions
 - ▶ Par exemple `slice(mois = jan, Temps) AND slice(ville = Berlin, Lieu)`
- ▶ Slice ne peut produire que des tranches du cube, du fait qu'une sélection n'est possible que sur une dimension.
- ▶ Dice peut produire des sous-cubes arbitraires.

Dice appliqué à deux dimensions



Dice appliqué à trois dimensions



- ▶ Jusqu'à présent, nous avons considéré des opérateurs appliqués à un seul cube.
- ▶ Dans un DW sont typiquement stockés plusieurs cubes, par exemple:
 - ▶ Un cube ayant les dimensions (Temps, Produit, Lieu) stockant le nombre de ventes par période, produit et magasin;
 - ▶ Un cube ayant les dimensions (Temps, Produit, Client) stockant le nombre de ventes en ligne par client et par période.

Pourquoi pas un cube (Temps, Produit, Lieu, Client)?

Cubes corrélés

29

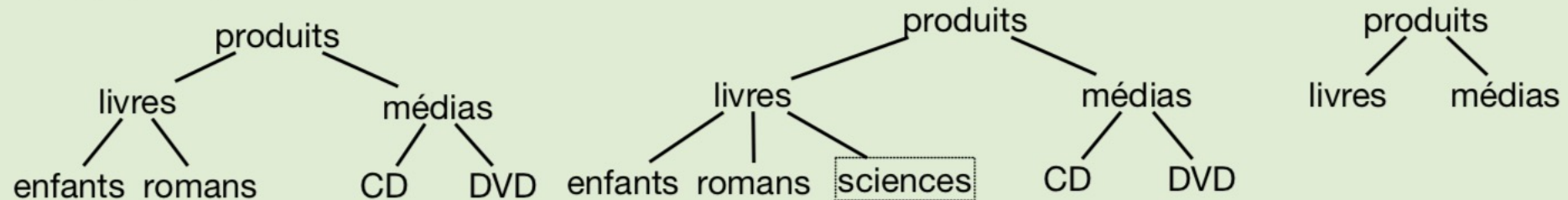
- Des **cubes corrélés** se partagent une ou plusieurs dimensions concordantes

Dimensions concordantes

Deux dimensions D et D' sont concordantes si, pour leurs domaines de valeurs, il'est vrai que $dom(D) \subseteq dom(D')$ ou $dom(D') \subseteq dom(D)$.

Exemples de dimensions concordantes et non-concordantes

concordantes



non-concordantes



Cubes corrélés

30

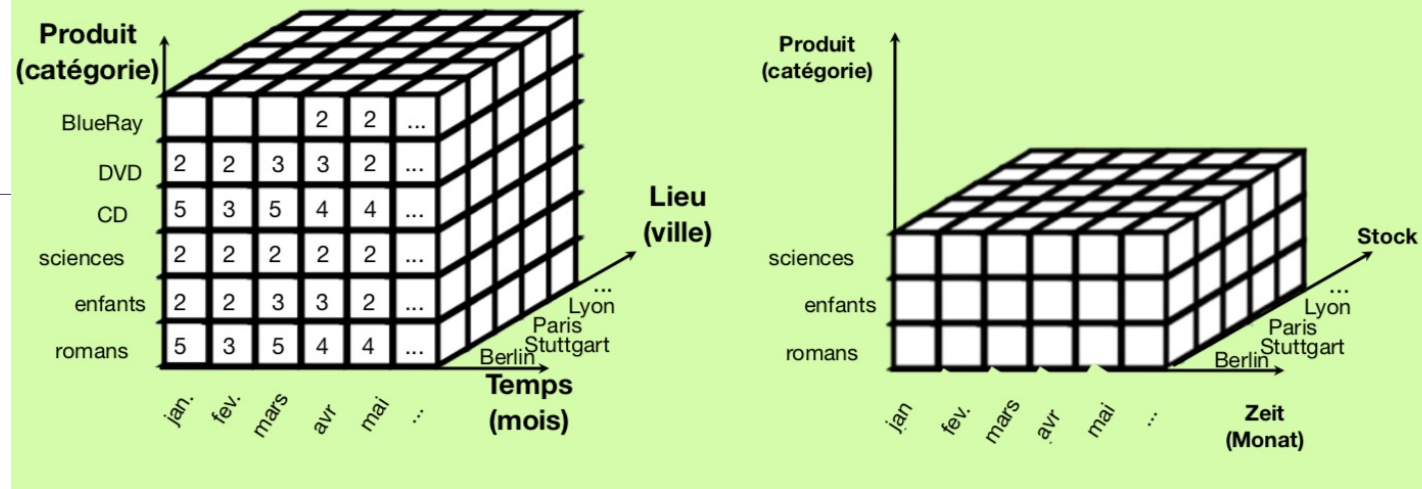
Cubes corrélés

Deux cubes $DC = (DS, M)$ et $DC' = (DS', M')$ sont **corrélés**, si au moins une paire de dimensions (D_i, D_j) , avec $D_i \in DS$, $D_j \in DS'$ est concordante.

Soit CD l'ensemble de toutes les paires

$\{(D_i, D_j) \mid D_i \in DS, D_j \in DS', D_i \text{ concordante avec } D_j\}$.

Exemple de deux cubes corrélés ayant $CD = \{(Produit, Produit), (Temps, Temps)\}$

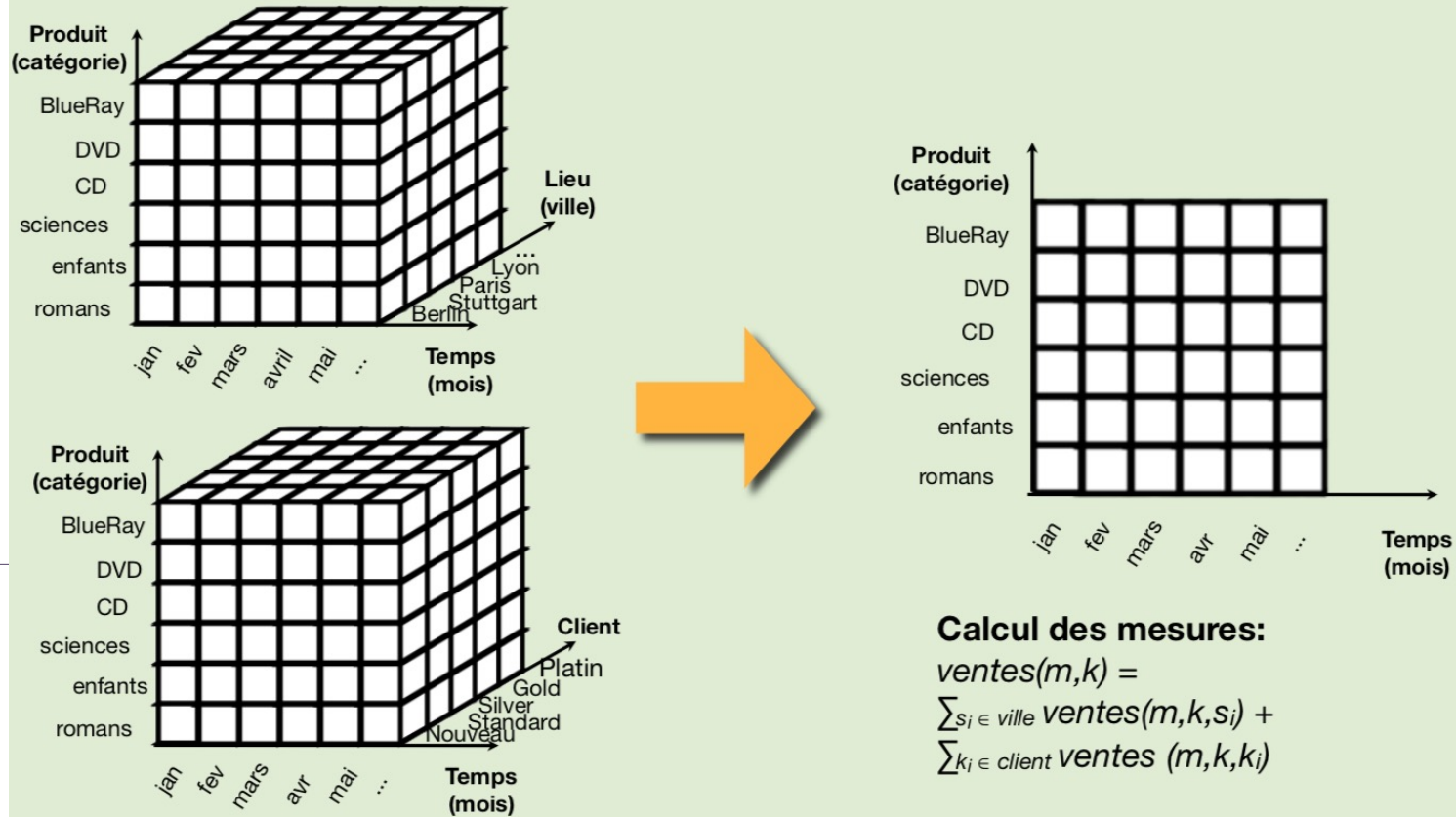


- ▶ **Drill-Across** est appliqué à plusieurs cubes corrélés.
 - ▶ Il sert à combiner les mesures des faits de différents cubes corrélés pour effectuer une analyse plus globale.
 - ▶ Hypothèse: la combinaison des mesures est raisonnable!
 - ▶ On peut par exemple déterminer les produits les plus populaires (nombres de ventes les plus élevés) en faisant la somme des mesures des deux cubes corrélés (Temps, Produit, Lieu) et (Temps, Produit, Client).
- ▶ Le cube produit par une opération *drill-across* appliquée aux cubes $DC = (DS, M)$ et $DC' = (DS', M')$ consiste en
 - ▶ L'ensemble des dimensions concordantes avec le moins de détails $\{D_i \mid (D_i, D_j) \in CD \text{ AND } D_i \subseteq D_j\}$ (voir transparent 32 pour la définition de CD)
 - ▶ L'ensemble des mesures $M \cup M'$ ou des mesures dérivées.

Drill-Across

32

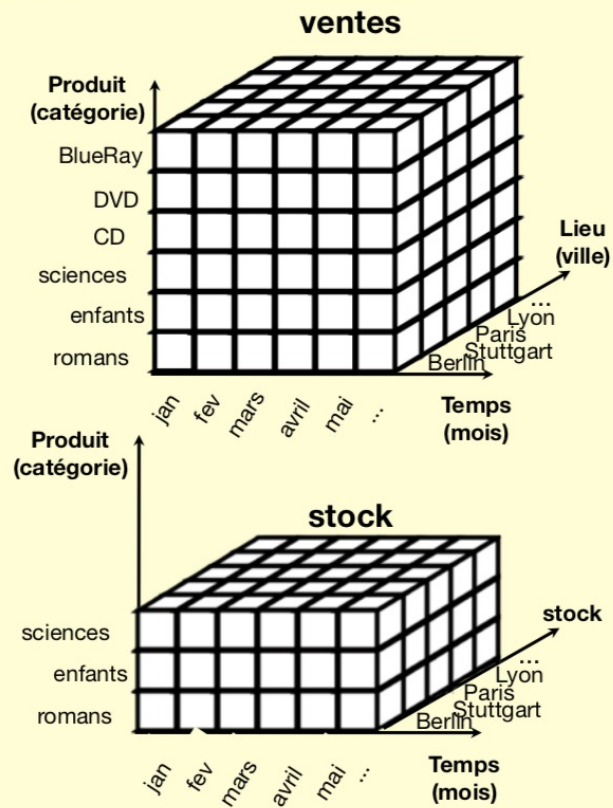
Drill-across exemple 1: Ventes totales (en magasin + en ligne) de produits par période



Drill-Across

33

Drill-across exemple 2: Pour quels produit le stock est toujours supérieur à 60% du nombre de ventes du même produit?



Opérateurs OLAP

34

- ▶ Les opérations OLAP vu jusqu'ici s'appliquent à des cubes multidimensionnels
 - ▶ Navigation dans une hiérarchie: *roll-up*, *drill-down*
 - ▶ Extension / réduction de la dimensionnalité: *split* / *merge*
 - ▶ Sélection: *slice*, *dice*
 - ▶ Combinaison de deux cubes: *drill-across*
- ▶ D'autres requêtes analytique appartenant aux opérations OLAP:
 - ▶ Calcul de moyenne glissante
 - ▶ Analyse portant sur des séries chronologiques
 - ▶ Comparaison relative de valeurs

Langages implémentant des opérateurs OLAP

35

- ▶ En général
 - ▶ Mise à disposition des opérations requises
 - ▶ Langages déclaratifs
 - ▶ Pas de requêtes SQL embriquées , mais verbose que le standard SQL
- ▶ Deux approches:
 - ▶ **Extensions SQL:**
Requêtes portant sur des données relationnelles stockées en schéma en étoile ou en schéma en flocon.
 - ▶ **Multidimensional Expressions (MDX):**
S'applique directement sur un modèle multidimensionnel (cubes, dimensions, faits, ...)

- Traduction d'un modèle multidimensionnel en un schéma en étoile ou en flocon
- Opérations avec **SQL-92**
 - *Slice / dice*: jointures et sélections
 - *drill-down*: jointures et projections
 - Agrégation d'un niveau: GROUP BY
- Extensions **OLAP** de SQL

Exemple

37

Exemple d'un schéma en étoile

Produit

PID	categorie	type
1	romans	livres
2	enfants	livres
3	sciences	livres
4	CD	médias
5	DVD	médias
6	BlueRay	médias

Temps

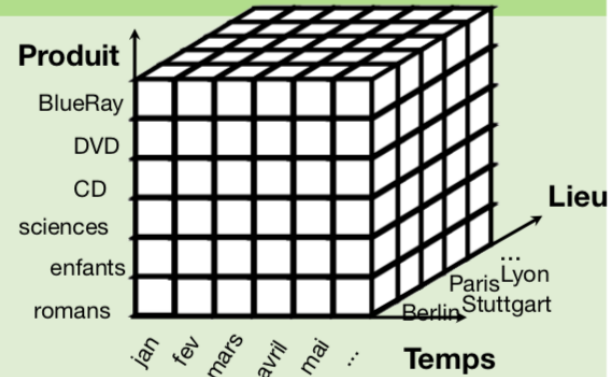
ZID	mois	trimestre	annee
1	Jan10	Q1 2010	2010
2	Fev10	Q1 2010	2010
...	...	...	...

Lieu

LID	ville
1	Berlin
2	Stuttgart
3	Paris
4	Lyon

Faits

PID	ZTD	LID	#Ventes	CD
1	1	1	5	30
1	1	2	5	37
1	1	3	5	45
1	1	4	5	20
2	1	1	2	33
2	1	2	2	35
2	1	3	2	40
2	1	4	2	35
...	...	...	...	...
1	2	1	3	22
...	...	...	...	...



Groupement multiple

38

► Rappel de GROUP BY

- GROUP BY <att1>, <att2>, ..., <attn> forme des partitions des données considérées par la requête en fonction des valeurs des attributs att1 à attn.
- Si GROUP BY est utilisé, peuvent apparaître dans la clause SELECT (i) les attributs att1 à attn plus (ii) des valeurs agrégées d'autres attributs.
- Agrégation avec les fonctions SUM, COUNT, AVG, MIN, MAX

Exemple d'une requête GROUP-BY - Total du chiffre d'affaire CD de livres par année, trimestre et mois?

```
SELECT    annee, trimestre, mois, SUM(CD)
FROM      Faits F, Temps T, Produit P
WHERE      P.PID = F.PID
AND        T.TID = F.TID
AND        P.type = 'livres'
GROUP BY  annee, trimestre, mois
```

Somme uniquement par mois
(association d'un mois avec
trimestre et année correspondant)

Pas de total de ventes par trimestre
ou année

Groupement multiple

39

Exemple d'une requête GROUP-BY - Total du chiffre d'affaire CD de livres par année, trimestre et mois?

```
SELECT  mois AS Time, SUM(CD) AS total_CD
FROM    Faits F, Temps T, Produit P
WHERE   P.PID = F.PID
AND     T.TID = F.TID
AND     P.type = 'livres'
GROUP BY mois
```

```
UNION
SELECT  trimestre AS Time, SUM(CD) AS total_CD
FROM    Faits F, Temps T, Produit P
WHERE   P.PID = F.PID
AND     T.TID = F.TID
AND     P.type = 'livres'
GROUP BY trimestre
```

```
UNION
SELECT  annee AS Time, SUM(CD) AS total_CD
FROM    Faits F, Temps T, Produit P
WHERE   P.PID = F.PID
AND     T.TID = F.TID
AND     P.type = 'livres'
GROUP BY annee
```

mois

trimestres

années

Time	total_CD
jan 2010	150
fev 2010	300
...	...
dec 2010	400
jan 2011	300
...	...
Q1 2010	500
...	...
Q4 2011	800
2010	2000
2011	2300

Groupement multiple

40

Résultat désiré -Total du CD par année, trimestre et mois

annee	trimestre	mois	CD
2010	Q1	jan	150
2010	Q1	fev	300
2010	Q1	mars	50
...	...	...	...
2011	Q1	jan	300
...	...	...	...
2010	Q1	⊥	500
...	...	...	...
2011	Q4	⊥	800
2010	⊥	⊥	2000
2011	⊥	⊥	2300

Utilisation d'une hiérarchie de dimension en SQL?

Exemple 2: CD de 2010 par (i) trimestre et type de produit et (ii) par trimestre et lieu.

trimestre	type	lieu	CD
Q1 2010	livres	⊥	50
...	livres	⊥	...
Q4 2010	livres	⊥	70
Q1 2010	médias	⊥	40
...	médias	⊥	...
Q4 2010	médias	⊥	60
Q1 2010	⊥	Berlin	60
...	⊥	Berlin	...
Q4 2010	⊥	Berlin	50
Q1 2010	⊥	Stuttgart	20
...	⊥	...	...

Agrégation pour plusieurs combinaisons de dimensions en SQL?

Groupement multiple

41

- ▶ Extensions SQL supportant le groupement multiple en une seule requête:
 - ▶ **GROUPING SETS** - Spécification explicite des combinaisons d'attributs de groupement
 - ▶ **CUBE** - groupement multidimensionnel
Génère toutes les combinaisons possibles des attributs spécifiés en paramètres
 - ▶ **ROLLUP** - groupement hiérarchique et multidimensionnel
- ▶ **Combinaisons** de ces clauses avec les clauses SQL connues (GROUP BY, HAVING, CASE, ...)

Groupement multiple: *Grouping Sets*

42

Grouping Sets

```
...  
GROUP BY GROUPING SETS ((<set1_a1>, ..., <set1_am>),  
                           (<set2_a1>, ..., <set2_an>),  
                           ...)
```

- ▶ Spécification explicite du groupement désiré
 - ▶ Chaque séquence d'attributs(<set_i_a₁>, ...) décrit un groupe désiré
- ▶ Equivalent à multiples requêtes GROUP BY unifiées et dont les attributs non-commun sont comblés avec des valeurs NULL ($\perp$).
- ▶ Il est possible d'identifier les attributs en fonction desquels le groupement d'un tuple est effectué en utilisant la fonction GROUPING(): GROUPING(<attr>) = 0 si groupement par <attr>, sinon 1.

Grouperement multiple: *Grouping Sets*

43

Exemple 2: CD de 2010 par (i) trimestre et type de produit et (ii) par trimestre et lieu.

trimestre	type	lieu	CD
Q1 2010	livres	⊥	50
...	livres	⊥	...
Q4 2010	livres	⊥	70
Q1 2010	médias	⊥	40
...	médias	⊥	...
Q4 2010	médias	⊥	60
Q1 2010	⊥	Berlin	60
...	⊥	Berlin	...
Q4 2010	⊥	Berlin	50
Q1 2010	⊥	Stuttgart	20
...	⊥	...	...

Exemple 2 en SQL en utilisant grouping sets

```
SELECT    trimestre,
          type,
          lieu,
          SUM(CD)AS CD
FROM      Faits F,
          Temps T, Produit P, Lieu L
WHERE     P.PID = F.PID
AND       T.TID = F.TID
AND       L.LID = F.LID
AND       T.annee = '2010'
GROUP BY GROUPING SETS (
          (trimestre, type),
          (trimestre, lieu)
```

Groupement multiple: *Grouping Sets*

44

Exemple 2 avec identification des attributs de groupe

trimestre	type	lieu	CD	GQ	GT	GL
Q1 2010	livres	⊥	50	0	0	1
...	...	⊥	...	0	0	1
Q1 2010	⊥	Berlin	60	0	1	0
...	⊥	Berlin	...	0	1	0

Exemple 2 avec identification des attributs du groupe en SQL

```
SELECT  trimestre, type, lieu, SUM(CD) AS CD
        GROUPING(trimestre) AS GQ, GROUPING(type) AS GT,
        GROUPING(lieu) AS GL
FROM    ...
WHERE   ...
GROUP BY GROUPING SETS ((trimestre, type), (trimestre, lieu))
```

Groupement multiple: *Grouping Sets*

45

► Sémantique de groupement de la clause GROUPING SETS

► **GROUP BY A, B** $\equiv$ **GROUP BY GROUPING SETS ((A, B))**

→ groupe (A, B)

► **GROUP BY GROUPING SETS ((A, B), (A, C), (A))**

$\equiv$

GROUP BY A, GROUPING SETS ((B), (C), ())

→ groupes (A) $\times$ ((B), (C), ()) = (A, B), (A, C), (A)

• **GROUP BY GROUPING SETS ((A, B), (B, C)),**

GROUPING SETS ((D, E), (D), ())

→ groupes ((A, B), (B, C)) $\times$ ((D, E), (D), ())

= (A, B, D, E), (A, B, D), (A, B), (B, C, D, E), (B, C, D), (B, C)

Groupement multiple

46

Spécifiez une requête SQL qui produit le résultat suivant

type	annee	ville	#ventes
livres	⊥	⊥	50
livres	2010	Berlin	30
livres	2010	Stuttgart	20
médias	⊥	⊥	40
médias	2010	Berlin	30
médias	2010	Stuttgart	10
⊥	⊥	Berlin	80
⊥	⊥	Stuttgart	30

Groupement multiple:

Groupement multidimensionnel

47

Groupement multidimensionnel en utilisant CUBE ()

```
...  
GROUP BY CUBE (<a1>, ..., <an>)
```

- ▶ Forme l'ensemble des parties de l'ensemble spécifié en paramètre
 - ▶ par exemple, CUBE(A, B, C) forme les groupes (A, B, C), (A, B), (A, C), (B, C), (A), (B), (C), ()
 - ▶ 2ⁿ groupes au total
- ▶ En combinaison avec une agrégation, CUBE () est utilisé pour calculer les valeurs agrégées de toutes les combinaisons.

Grouperement multiple:

Grouperement multidimensionnel

48

CD par (toutes les combinaisons de) trimestre, type de produit et lieu en 2010

trimestre	type	lieu	total_CD
Q1 2010	livres	Berlin	10
Q1 2010	livres	Stuttgart	20
...	...	...	...
Q1 2010	livres	⊥	20
...	...	⊥	...
Q1 2010	⊥	Berlin	80
Q1 2010	⊥	Stuttgart	50
...	⊥	...	...
⊥	livres	Berlin	50
⊥	livres	Stuttgart	80
⊥	...	...	...
Q1 2010	⊥	⊥	160
...	⊥	⊥	...
⊥	livres	⊥	200
⊥	médias	⊥	300
⊥	⊥	Berlin	200
⊥	⊥	Stuttgart	100
⊥	⊥	...	...
⊥	⊥	⊥	1000

Code SQL livrant ce résultat

```
SELECT  trimestre, type, lieu,
        SUM(CD) AS total_CD
FROM    Faits F,
        Temps T, Produit P, Lieu L
WHERE   P.PID = F.PID
AND     T.TID = F.TID
AND     L.LID = F.LID
AND     T.annee = '2010'
GROUP BY CUBE (trimestre, type, lieu)
```

Groupement multiple:

Groupement multidimensionnel

49

- Sémantique de groupement de l'opérateur CUBE

- **GROUP BY CUBE(A, B, C)**

≡

GROUP BY GROUPING SETS ((A,B,C), (A,B), (A,C),
(B,C), (A), (B), (C), ())

- **GROUP BY CUBE(A, B), CUBE(B, C)**

→ groupes ((A, B), (A), (B), ()) × ((B, C), (B), (C), ())

= (A, **B**, **B**, C), (A, **B**, **B**), (A, B, C), (A, B),

(A, B, C), (A, B), (A, C), (A),

(**B**, **B**, C), (**B**, **B**), (B, C), (B),

(B, C), (B), (C), ()

= (A, B, C), (A, B), (A, B, C), (A, B),

(A, B, C), (A, B), (A, C), (A),

(B, C), (B), (B, C), (B),

(B, C), (B), (C), ()

Duplicata de combinaisons
(e.g., (A, B, C), (A, B), (B), ...)
ne sont **pas** éliminées!

- **GROUP BY GROUPING SETS(CUBE(A, B), CUBE(B,C))**

≡

GROUP BY GROUPING SETS ((A,B), (A), (B), (), (B,C), (B), (C), ())

Groupement multiple:

Groupement multidimensionnel

50

Spécification d'une requête SQL utilisant l'opérateur CUBE

Ce tableur représente le nombre de DVDs vendus le premier trimestre 2010, agrégé par les différentes combinaisons de mois et de lieu. Spécifiez la requête SQL produisant ce résultat.

	jan	fev	mars	Total
Berlin	20	25	22	67
Stuttgart	10	15	17	42
Paris	30	25	30	85
Lyon	10	12	10	32
Total	70	77	79	226

Groupement multiple:

Groupement hiérarchique multidimensionnel

51

Groupement multidimensionnel utilisant ROLLUP ()

...
GROUP BY ROLLUP (<a₁>, ..., <a_n>)

- ▶ En SQL-92, on peut formuler une UNION de k sous-requêtes pour aboutir à un groupement de k niveaux d'une dimension.
 - ▶ Laborieux à spécifier, à traiter (k lectures de la table des faits) et ordre défavorable à la présentation (et tri difficile).
- ▶ ROLLUP calcule une agrégation pour plusieurs niveaux d'une hiérarchie en ordre orienté favorable aux analyses.
 - ▶ Attributs <a₁>, ..., <a_n> sont issue d'un même schéma de dimension et sont spécifiés en ordre croissant de granularité.

Grouperement multiple:

Grouperement hiérarchique multidimensionnel

52

CD de livres par année, trimestre et mois.

annee	trimestre	mois	total_CD
2010	Q1	jan	150
2010	Q1	fev	300
2010	Q1	mars	50
...	...	...	...
2011	Q1	jan	300
...	...	...	...
2010	Q1	⊥	500
...	...	...	...
2011	Q4	⊥	800
2010	⊥	⊥	2000
2011	⊥	⊥	2300
⊥	⊥	⊥	4300

Requête SQL utilisant ROLLUP

```
SELECT  année, trimestre, mois,
        SUM(CD)AS total_CD
FROM    Faits F,
        Temps T, Produit P
WHERE   P.PID = F.PID
AND     T.TID = F.TID
AND     P.type = 'livres'
GROUP BY ROLLUP (annee, trimestre, mois)
```


Groupement multiple:

Groupement hiérarchique multidimensionnel

53

- ▶ Sémantique de groupement de l'opérateur ROLLUP

- ▶ **GROUP BY ROLLUP**(A1, A2, ..., An)

≡

GROUP BY

GROUPING SETS ((A1, ..., An-1, An), (A1, ..., An-1), ..., (A1, A2), (A1), ())

ou s'appliquent les dépendances fonctionnelles $A_n \rightarrow A_{n-1} \rightarrow \dots \rightarrow A_2 \rightarrow A_1$

- ▶ **GROUP BY ROLLUP**(A1, ..., An), **ROLLUP**(B1, ..., Bm)

- groupes ((A1, ..., An), ..., (A1, A2), (A1), ())

×

((B1, ..., Bn), ..., (B1, B2), (B1), ())

Groupement multiple:

Groupement hiérarchique multidimensionnel

54

Spécifiez une requête SQL utilisant ROLLUP

La requête SQL calcule le nombre de ventes en décembre de produits de type médias regroupé par année, mois, catégorie et ville.

Le résultat est représenté ci-contre.

année	mois	categorie	ville	ventes
2010	dec	CD	-	200
2010	dec	DVD	-	150
2010	dec	BlueRay	-	50
2010	dec	-	-	400
2011	dec	-	-	450
2010	-	-	-	2000
2011	-	-	-	3000
-	-	-	-	5000
2010	-	-	Berlin	1000
2010	-	-	Stuttgart	300
2010	-	-	...	...
2011	-	-	Berlin	1500
2011	-	-	...	...
-	-	-	Berlin	4000
-	-	-	Stuttgart	2000
-	-	-	...	...

MDX

- ▶ MDX = **Multidimensional Expressions**:
Proposition de Microsoft “OLE DB for OLAP”
- ▶ Langage de requête s'appliquant sur le MDDM
 - ▶ Standard **sans sémantique fixe** (définie par des exemples)
 - ▶ Concepts MDDM (dimensions, mesures) manipulés directement
 - ▶ Requêtes plus compacte que les requêtes SQL équivalentes
 - ▶ Syntaxe similaire à SQL (mais sémantique différente)
 - ▶ Langage très complexe et puissant
- ▶ Création des objets à manipuler (DDL) se fait par d'autres moyens: DSO Interface (Decision Support Objects) de SQL Server
- ▶ Est utilisé par beaucoup d'outils commerciaux pour communiquer avec des BD OLAP.

- ▶ Le langage des expressions multidimensionnelles possède une syntaxe appropriée à l'interrogation et manipulation des données multidimensionnelles mémorisées dans un cube OLAP1.
- ▶ Bien qu'il soit possible de traduire certaines expressions dans le langage SQL traditionnel, cela nécessite une syntaxe SQL souvent maladroite même pour des expressions MDX très simples.
- ▶ MDX a été adopté par une large majorité de fournisseur de la technologie OLAP et est devenu un standard de facto pour les systèmes OLAP.

MDX Principe de Base et Structure

58

- ▶ Une requête MDX retourne un rapport à plusieurs dimensions
 - ▶ par exemple un tableur 2D
 - ▶ par exemple plusieurs tableurs 2D pour représenter trois dimensions
- ▶ La structure d'une requête MDX est identique à la structure des clauses SELECT-FROM-WHERE en SQL
 - ▶ Mais la sémantique est différente

Structure générale d'une requête MDX

```
SELECT    <axis-spec1>, <axis-spec2>, ...  
FROM      <cube-spec1>, <cube-spec2>, ...  
WHERE     ( <selection-specification> )
```

- ▶ Exemple:

```
SELECT Measures.[Unit Sales] ON COLUMNS,  
       [Store].[All Stores] ON ROWS  
FROM [Sales]
```

* Les délimiteurs [et] peuvent ne pas être mis si pas d'ambiguïté.

MDX : Principe de Base et Structure

59

Exemple simple d'une requête MDX (cube & dimensions voir transparent 3)

```
SELECT { Berlin, Stuttgart } ON ROWS
       { [1er trimestre], [2nd trimestre].CHILDREN } ON COLUMNS
FROM   CubeVentes
WHERE  ( Measures.Ventes,
        Temps.[2010],
        Produit.Produit )
```

Aggregation de la mesure
(*measure*) ventes en utilisant
fonction prédefinie (e.g., SUM)

Sélection sur la dimension Temps
(uniquement la période 2010)

Sélection des produits, ici, le
niveau TopD est sélectionné

	1er trimestre 2010	avril 2010	mai 2010	juin 2010
Berlin	valeurs numériques			
Stuttgart				

► Parenthèses en MDX

- {} - Ensemble des éléments servant à la création d'une dimension du résultat de la requête
- [] - Représentation d'espaces, de caractères spéciaux et d'interprétation non numérique de chiffres.
- () - Sélection de tuples dans la clause WHERE

► SELECT - description des axes du cube résultat

- Chaque dimension du résultat est associé à un rôle correspondant à sa représentation dans le tableur retourné par la requête MDX
ON COLUMNS, ON ROWS, ON PAGES, ON SECTIONS, ON CHAPTERS
- Une dimension du résultat correspond à une (sélection sur une) dimension du MDDM sur un ou plusieurs niveaux de la hiérarchie.
{Berlin, Stuttgart} de la dimension Lieu, niveau Ville
{[1er trimestre], [2nd trimestre].CHILDREN} de la dimension Temps, niveaux trimestre et mois.

► FROM - Spécification du cube

- Ensemble de cubes étant nécessaire à la création du cube résultat.
- Si plusieurs cubes y sont spécifiés, la sémantique implique une **jointure multidimensionnelle**. Dans ce cas, chaque paire de cubes doit posséder au moins une dimension concordante.

► WHERE - Restriction

- Restrictions appliquées sur les cubes spécifiés dans la clause FROM.
 - Spécification des restrictions par une liste de **nœuds de la hiérarchie** d'une dimension nommée *slicer-dimension*.
-
- Les mesures sont des éléments « normaux » de la dimension spéciale des mesures nommée **Measures** (celle-ci peut être utilisée dans les clauses WHERE et FROM).

Expressions de collections

62

- ▶ **Enumération** de nœuds d'une hiérarchie dimensionnelle
 - ▶ Des nœuds individuels ne proviennent pas nécessairement du même niveau
 - ▶ Par exemple, `{romans, enfants, [médias]}` retourne des catégories et un type de la dimension Produit.
- ▶ **Expressions donnant un ensemble de nœuds**
 - ▶ **CHILDREN**: enfants d'un nœud de la hiérarchie
e.g., `[2nd trimestre].CHILDREN = {avril, mai, juin}`
 - ▶ **MEMBERS**: nœuds d'un niveau
e.g., `Temps.Trimestre.MEMBERS = {[1er trimestre], ..., [4ème trimestre]}`
 - ▶ **PARENT**: Parent d'un nœud
e.g., `romans.PARENT = {livres}`
 - ▶ ...

Expressions de collections

63

► Emboitage de collections avec CROSSJOIN

- Projection de deux dimensions en une seule
- Sémantique: produit Cartésien

Exemple CROSSJOIN

```
SELECT  CROSSJOIN({Berlin, Stuttgart}
              {kiosque, [grande surface]}) ON ROWS
              { 2010.CHILDREN } ON COLUMNS
FROM    CubeVentes
WHERE    ( Measures.Ventes, Temps.[2010], Produit.[médias] )
```

		1er trimestre 2010	2nd trimestre 2010	3ème trimestre 2010	4ème trimestre 2010
Berlin	kiosque				
	grande surface	valeurs numériques (nombre de ventes de produits de type médias en 2010)			
Stuttgart	kiosque				
	grande surface				

- ▶ **Sélection relative** par rapport à la position d'un nœud de l'instance d'une dimension
 - ▶ LASTCHILD - dernier enfant d'un nœud
e.g. `livres.LASTCHILD = {sciences}`
 - ▶ NEXTMEMBER - prochain voisin (à droite) au même niveau
e.g. `[1er trimestre 2010].NEXTMEMBER = {[2nd trimestre 2010]}`
 - ▶ LEAD(*x*) - voisin à distance *x* positive (à droite) au même niveau
e.g. `[novembre 2010].LEAD(2) = {[janvier 2011]}`
 - ▶ [*x*]:[*y*] - Restriction du domaine entre *x* et *y*
e.g. `[1992]:[2002]` restreint la période à considérer aux années 1992 à 2002.

Expressions de collections

65

- ▶ Fonctions appliquées au **schéma d'une dimension** exploitant l'ordre défini de cette structure.
 - ▶ LEVEL - identification du niveau schématique
e.g. `romans.LEVEL = {[catégorie]},`
`[médias].LEVEL = {type}`
 - ▶ LEVELS(x) - exploitation de la numérotation interne des niveaux du schéma de la dimension pour accéder au niveau x
e.g. `Temps.LEVELS(1) = {[année]}`

Conditions de Production du Résultat

66

- ▶ Construction du cube résultat en fonction de résultats calculés par la requête.
- ▶ Requêtes top-k
 - ▶ TOPCOUNT
 - ▶ TOPRECENT
 - ▶ TOPSUM
- ▶ Filtre
 - ▶ FILTER

Conditions de Production du Résultat

67

Exemple de la fonction TOPCOUNT

Totaux des ventes des cinq régions françaises ayant vendu le plus en 2010 (Hypothèse: hiérarchie Pays-Région-Ville)

```
SELECT    {Measures.Ventes} ON COLUMNS
           {TOPCOUNT(France.CHILDREN, 5, Measures.Ventes)}
           ON ROWS
FROM      CubeVentes
WHERE      ( Measures.Ventes, [2010] )
```

Exemple de la fonction Filter

Totaux des ventes de produits de type médias des régions dont le total des ventes de 2002 est supérieur à celui de 2001. Ici, le total des ventes est retourné par trimestre de 2002.

```
SELECT    {FILTER(France.CHILDREN,
                  ([2002], Ventes) > ([2001], Ventes) ) } ON COLUMNS
           {trimestre.MEMBERS } ON ROWS
FROM      CubeVentes
WHERE      ( Ventes, [2002], Produit.[médias] )
```


► Fonctions sur l'axe du temps

► `PERIODSTODATE(k, d)`

Tous les nœuds de la classe k jusqu'à la date d

e.g., `PERIODSTODATE(trimestre, [15/11/2002]) = 1/10/2002 - 15/11/2002`

► `LASTPERIODS(k, d)`

k derniers laps de temps précédant la date d (k et d au même niveau).

e.g., `LASTPERIODS(3, [sept. 2002]) = {[juin 2002], [juillet 2002], [août 2002]}`

► `PARALLELPERIOD(k, n, d)`

e.g., `PARALLELPERIOD([Année], 3, [sept. 2002]) = {[sept. 1999]}`

Autres fonctions

- ▶ Covariance
- ▶ Corrélation
- ▶ Moyenne glissante
- ▶ ...

▶ Opérations OLAP

- ▶ Navigation dans les dimensions: Roll-Up, Drill-Down
- ▶ Augmentation / réduction de la dimensionalité: Split / Merge
- ▶ Sélection: Slice, Dice
- ▶ Combinaison de cubes corrélés: Drill-Across

▶ Langages OLAP

- ▶ MDX - concepts du MDDM et de tableurs
- ▶ Extensions SQL: GROUPING SETS, CUBE, ROLLUP